Project Phase B - Part 2

In Part 1, we built a linear AI pipeline. Data entered, moved step-by-step, and an email was sent. It was predictable and static.

Real-world systems—especially disaster management systems—don't work like that. They require **dynamic routing**, **integration with traditional Machine Learning**, **Human-in-the-Loop oversight**, and the ability to **learn from failure**.

Disaster Management System

Objective:

Develop an **Autonomous Disaster Management System** as a **state-driven python application** (not merely a sequence of prompts) that integrates all below mentioned system components. The system should accept a geographic location as input, predict potential weather-related disasters using an ML model, route the situation to the appropriate AI module, request human approval before issuing alerts, and continuously improve its performance by learning from rejected outputs.

The System Components

Your architecture will consist of the following distinct nodes. You must figure out the correct sequence, how to pass complex data (like arrays and dataframes) between them, and where to place conditional branches.

1. Environmental Data Ingestion

- Uses a Weather API to pull current environmental conditions and appends this fresh data to a historical dataset.

2. Time Series Forecasting

- Repurposes the time-series forecasting concepts you learned in Part 1. It analyzes the historical data to forecast near-future weather metrics (e.g., rainfall volume over the next 48 hours).

3. Disaster Prediction Model

- A traditional machine learning classification model. It takes the newly forecasted metrics and predicts the likelihood and specific type of a weather-related disaster (e.g., Flood, Hurricane, Heatwave).

***Note**: Don't get into maths and working of these models yet, we have it covered under further plans. As of now focus on integration of pretrained models.*

4. News Monitoring & Context

- Concurrently triggers an agent to search a News API for local reports. This provides real-world, localized context to back up the ML prediction.

5. Cognitive Assessor & Router

- The "brain" of the operation. An LLM evaluates the ML prediction alongside the live news context to assess the severity level. Based on this severity, it **routes** the data to the appropriate specialized department.

6. The Department Agents

- Three specialized agents (Emergency Response, Civil Defense, and Public Works). Depending on where the Router sends the data, the selected agent will draft a highly specific action plan and alert message.

7. Human-in-the-Loop Gatekeeper

- Pauses the entire workflow. The drafted alert is presented to a human user. The system waits for the human to either "Approve" (which sends the email alert and ends the process) or "Reject" (which triggers the learning phase).

8. The Self-Improving Loop

- If the human rejects the output and provides feedback, the agent doesn't just try again blindly. It reflects on the failure, generates a permanent "Insight/Rule" (e.g., *Always include emergency contact numbers for high-severity alerts*), updates its own prompt, and attempts the generation again.

Deliverables

1. **The Graph Architecture:** A fully functional, state-driven pipeline that successfully connects all the components above using conditional logic.
2. **The ML-LLM Bridge:** Proof that you can seamlessly pass numerical outputs from standard Python ML models directly into LLM reasoning prompts.
3. **The Memory & Adaptation:** A demonstrable Self-Improving Loop. You must show that when an agent's output is rejected by a human, it successfully alters its next output based on newly generated insights.
4. **Frontend:** Make a decent and minimal frontend having basic UI to showcase your project.
